

Integral Projection Model

Loïc Pages & Eric Imbert

2026-04-09

Import models for vital rates (see ModelSelectionReport.Rmd)

Produce a global IPM model

See IPM_Pop_Year.Rmd for estimations per population and per year

Initialisation

Creation of the functions for the kernels.

```
annees <- 1995:2021
populations <- c("Po", "Au", "Pe", "E1", "E2", "Cr")

#Fecundity - Number of capitulas
numbofcap <- function(x, a) {
  fake_data <- expand.grid( #Create a false data set with all combinations of variables
    year = annees,
    Pop = populations,
    Size0Mars = x,
    Age = a)
  predi <- exp(predict(Fecundity, fake_data)) #Predicted number of capitulas
  meanpredi <- aggregate(predi, list(fake_data$Size0Mars), mean) #Make the prediction data more usable,

  # meanpredi$V1[meanpredi$Group.1<5] <- 0 #Nbr of capitulas=0 if size<5 cm
  nbcap <- ifelse(meanpredi$V1>=0, meanpredi$V1, 0) #Avoid nbr of capitulas=0
  return(matrix(nbcap, length(x), length(x), byrow = T))
}

#Survival Probability for age 1
sx01 <- function(x,a) {
  fake_data <- expand.grid(
    year = annees,
    Pop = populations,
    Size0Mars = x,
    Age = a)
  predi <- predict(Seedling_surv, fake_data, allow.new.levels = T)
  meanpredi <- aggregate(predi, list(fake_data$Size0Mars), mean)
  return(meanpredi$V1)
}

#Survival Probability for age > 1
sx02 <- function(x, a) {
  fake_data <- expand.grid(
    year = annees,
```

```

Pop = populations,
Size0Mars = x,
Age = a)
predi <- predict(Plant_surv, fake_data, allow.new.levels = T)
meanpredi <- aggregate(predi, list(fake_data$Size0Mars), mean)
return(meanpredi$V1)
}

#Flowering Probability

#Flowering function for The survival-growth kernel
flr0 <- function(x, a, Beta) {
  fake_data <- expand.grid(
    year = annees,
    Pop = populations,
    Size0Mars = unique(x),
    Age = a)
  predi <- plogis(predict(Flowering, newdata = fake_data, allow.new.levels = T, type = "link") + Beta)
  meanpredi <- aggregate(predi, list(fake_data$Size0Mars), mean)
  return(meanpredi$V1)
}

#Flowering function for fecundity fyx0 - same function the difference is in the format of the output
flr1 <- function(x, a, Beta) {
  fake_data <- expand.grid(
    year = annees,
    Pop = populations,
    Size0Mars = unique(x),
    Age = a)
  mu <- plogis(predict(Flowering, fake_data, allow.new.levels = T, type = "link") + Beta)
  mu2 <- aggregate(mu, list(fake_data$Size0Mars), mean)
  return(matrix(mu2$V1, MatrixDim, MatrixDim, byrow = T))
}

# Growth function (using cumulative distribution function)
#Transformed Gaussian
Gyx <- function(ysup,yinf,x,a){
  sizepow <- 0.4343354
  yinfpow <- yinf**sizepow
  ysuppow <- ysup**sizepow
  yinfpow[1] <- -Inf
  ysuppow[length(ysuppow)] <- Inf

  fake_data <- expand.grid(
    year = annees,
    Pop = populations,
    Size0Mars = unique(x),
    Age = a)
  predi <- predict(Growth, fake_data, allow.new.levels = T)
  meanpredi <- aggregate(predi, list(fake_data$Size0Mars), mean) #Make the prediction data more usable,
  resvar <- predict(Growth$resid_fit, newdata = fake_data, allow.new.levels = T)
  meanresvar <- aggregate(resvar[,1], list(fake_data$Size0Mars), mean)$x

```

```

cdf.sup <- mapply(pnorm, mean = meanpredi$V1, sd = sqrt(meanresvar), MoreArgs = list(q=ysuppow))
cdf.inf <- mapply(pnorm, mean = meanpredi$V1, sd = sqrt(meanresvar), MoreArgs = list(q=yinfpow))
return(cdf.sup - cdf.inf)
}

#Seedling size (using cumulative distribution function)
densSeedl <- function(ysup, yinf) {
  yinf[1] <- -Inf
  ysup[length(ysup)] <- Inf
  fake_data <- expand.grid(
    year = annees,
    Pop = populations)

  predi <- predict(Seedling_size, newdata = fake_data)[, 1]
  meanpredi <- mean(predi)
  shape <- 1 / Seedling_size$phi
  scale <- meanpredi * Seedling_size$phi
  cdf.sup <- pgamma(ysup, shape = shape, scale = scale)
  cdf.inf <- pgamma(yinf, shape = shape, scale = scale)
  return(cdf.sup - cdf.inf)
}

#Establishment rate
Estbli <- function(Pop=populations, year=annees){
  fake_data <- expand.grid(
    year = year,
    Pop = Pop,
    Capitule = 1:200)
  plt <- predict(Estbrate, newdata = fake_data)[,1]
  mu <- mean(plt/fake_data$Capitule)
  return(mu)
}

#Combine fecundity and flowering probability
fyx0 <- function(x , a, Estbl, Beta = 0) {
  p.flow <- flr1(x, a, Beta) #flowering probability
  n.captl <- numbofcap(x, a) #number of capitula per flowering plant
  Estbl <- Estbli(populations,annees)

  feco <- p.flow * n.captl * Estbl
  return(feco)
}

n.size <- MatrixDim

#Discretization of vital rates function and creation of discretized matrices representing the kernel
K.fnc0 <- function(age, n.size, estbl, Beta=0){

  minsize <- 0.5
  maxsize <- 25
  maxsize1 <- 15
  h <- (maxsize - minsize) / n.size
  h1 <- (maxsize1 - minsize) / n.size

```

```

b <- minsize + c(0:n.size) * h

#sizes for an individual of age a
#midpoint
ymid.a = 0.5 * (b[1:n.size] + b[2:(n.size + 1)])
#intervals
yI.a <- matrix(nrow = n.size, ncol = 2)
for (i in 1:n.size){
  yI.a[i,1] <- b[i]
  yI.a[i,2] <- b[i+1]
}

b1 = minsize + c(0:n.size) * h1
#sizes for an individual of age 1
#midpoint
ymid.1 = 0.5 * (b1[1:n.size] + b1[2:(n.size + 1)])
#intervals
yI.1 <- matrix(nrow = n.size, ncol = 2)
for (i in 1:n.size){
  yI.1[i,1] <- b1[i]
  yI.1[i,2] <- b1[i+1]
}

if(age==1){h <- h1
  ymid.a <- ymid.1
}

#The growth kernel (matrix 50*50 with size at t+1 on rows and size at t on columns)
G <- Gyx(ysup=yI.a[,2], yinf=yI.a[,1], x=ymid.a, a=age)

#The survival-growth kernel #remove flowering plants since they died
sx0 <- ifelse(age==1,sx01,sx02)
if(age==1){surv <- sx01(ymid.a,1)}
if(age!=1){surv <- sx02(ymid.a,age)}
S_flr <- surv*(1-flr0(ymid.a,age,Beta)) #list of probability of surviving and not flowering for all s

Ps <- sweep(G, MARGIN=2, S_flr, '*') #We multiply each column of G by S_flr
P <- matrix(Ps, nrow=n.size)

#Fecundity kernel
ProporSeedlSize <- densSeedl(ysup=yI.1[,2],yinf=yI.1[,1])
feco <- fyx0(x = ymid.a, a = age, Estbl = estbl, Beta = Beta)
# feco <- feco + 3.33/8

Fec <- sweep(feco, MARGIN=1, ProporSeedlSize, '*')

IPMk <- P + Fec
return(list(M=IPMk,Pk=P,Fk=Fec))
}

```

the super matrix and growth rate

```

super_matrix = function(Beta_test){
  Pk <- matrix(data = NA, nrow = MatrixDim, ncol = MatrixDim)

```

```

Fk <- matrix(data = NA, nrow = MatrixDim, ncol = MatrixDim)
Mk <- matrix(data = NA, nrow = MatrixDim, ncol = MatrixDim)
Kernels <- array(data = 0, dim = c(AgeMax, 3, MatrixDim, MatrixDim))
GlobMatrIPM <- array(0, dim = c((AgeMax * MatrixDim), (AgeMax * MatrixDim)))

for (a in (1:(AgeMax - 1))){
  Kls <- K.fnc0(a, MatrixDim, estbl, Beta_test)
  Kernels[a, 1, , ] <- Kls$M
  Kernels[a, 2, , ] <- Kls$Pk
  Kernels[a, 3, , ] <- Kls$Fk
  # print(lambda(Kernels[a, 1, , ]))
  debut <- (a - 1) * MatrixDim + 1
  fin <- debut + (MatrixDim - 1)
  GlobMatrIPM[1:MatrixDim, (debut:fin)] <- Kls$Fk
  GlobMatrIPM[((fin + 1):(fin + MatrixDim)), (debut:fin)] <- Kls$Pk
}

Kls <- K.fnc0(AgeMax, MatrixDim, estbl, Beta_test)
Kernels[AgeMax, 1, , ] <- Kls$M
Kernels[AgeMax, 2, , ] <- Kls$Pk
Kernels[AgeMax, 3, , ] <- Kls$Fk
# print(lambda(Kernels[a,1,,]))
debut <- (AgeMax - 1) * MatrixDim + 1
fin <- debut + (MatrixDim - 1)
GlobMatrIPM[1:MatrixDim, (debut:fin)] <- Kls$Fk
GlobMatrIPM[(debut:fin), (debut:fin)] <- Kls$Pk

return(GlobMatrIPM)
}

Global_IPM <- super_matrix(0)
print(lambda(Global_IPM))

## [1] 0.898229
save(Global_IPM,file="Global_IPM.RData")

Test number of classes in the integration
lambk <- NULL
library(foreach)

##
## Attachement du package : 'foreach'

## Les objets suivants sont masqués depuis 'package:purrr':
##
##      accumulate, when
library(doParallel)

## Le chargement a nécessité le package : iterators
## Le chargement a nécessité le package : parallel
cl <- makeCluster(10)
registerDoParallel(cl)

```

```

lambk <- foreach (k=seq(2,50,5),.packages=c("splines","spaMM","popbio")) %dopar% {
  MatrixDim <- k
  obs_lambda <- lambda(super_matrix(0))
  obs_lambda
}
stopCluster(cl)

lambk <- unlist(lambk)
plot(x = (1:length(lambk))*5, y = lambk, type = "l", lwd=2, xlab = "Number of intervals",ylab="Growth rate")

```

